DHA Suffa University
Department of Computer Science
Artificial Intelligence Program (CS-2001L)
Fall 2026

Lab 02 – Data Structures and Functions

Objectives

- Store and manage collections of data using lists, tuples, dictionaries, and sets
- Write short and clean list comprehensions
- Create reusable functions with parameters, default values, and return values
- Use lambda (one-line) functions
- Build a **Student Record Manager** using a dictionary and functions

Tools required: Google Colab or Jupyter Notebook

Part 1: Lists

A list stores many values in one variable. Lists are **ordered**, **mutable** (changeable), and allow **duplicates**. They are created with square brackets.

Example 1:

```
languages = ["Python", "Java", "C++"]

for language in languages:
    print(language)

if "Python" in languages:
    print("Python is included in the list")
```

Example 2:

```
products = ["Laptop", "Mouse", "Keyboard", "Monitor", "Headphones"]
stock = [12, 45, 8, 0, 20]

print("PRODUCT INVENTORY REPORT")
print("-" * 40)

for i in range(len(products)):
```

```
    if stock[i] == 0:
        status = "Out of Stock"
    elif stock[i] < 10:
        status = "Low Stock"
    else:
        status = "Available"

    print(f"{products[i]:<15} | Quantity: {stock[i]:>3} | {status}")

print("-" * 40)

total_products = len(products)
out_of_stock = stock.count(0)
low_stock = sum(1 for quantity in stock if 0 < quantity < 10)

print(f"Total product types: {total_products}")
print("Out of stock: ", out_of_stock)
print(f"Low stock: {low_stock}")
```

Indexing and slicing

Indexing starts from **0**. Negative indexes count from the end.

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])       # 10 (first)
print(numbers[-1])      # 50 (last)
print(numbers[1:4])     # [20, 30, 40] (end index is not included)
print(numbers[:3])      # [10, 20, 30]
print(numbers[2:])      # [30, 40, 50]
```

Common list methods

Method	Purpose	Example
`append(x)`	Add an item at the end	`numbers.append(60)`
`insert(i, x)`	Add an item at position `i`	`numbers.insert(1, 15)`
`remove(x)`	Delete the first item equal to `x`	`numbers.remove(30)`
`pop()`	Remove and return the last item (or the item at an index)	`numbers.pop()`
`sort()`	Sort the list in place	`numbers.sort()`
`reverse()`	Reverse the list in place	`numbers.reverse()`
`len(list)`	Number of items	`len(numbers)`

```
marks = [70, 85, 60]
marks.append(90)          # [70, 85, 60, 90]
marks.insert(0, 55)       # [55, 70, 85, 60, 90]
marks.remove(60)          # [55, 70, 85, 90]
last = marks.pop()        # last = 90, marks = [55, 70, 85]
marks.sort()              # [55, 70, 85]
print(marks, last)
```

Looping through a list

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
    print(fruit)

if "banana" in fruits:
    print("We have bananas")
```

Task 1.1

1. Create a list of 5 numbers. Print the first item, the last item, and the middle three items
2. Add a new number at the end, remove one number, and sort the list
3. Use a loop to find the **sum** and the **largest** number in the list (without using `sum()` or `max()`)

Part 2: Tuples

A tuple is like a list, but it is **immutable**: it cannot be changed after it is created. Tuples use parentheses.

```
point = (10, 20)
days = ("Mon", "Tue", "Wed")

print(point[0])         # 10
print(len(days))        # 3

# point[0] = 5          # ERROR: a tuple cannot be changed
```

Tuple unpacking

```
x, y = point
print(x, y)             # 10 20
```

When to use a tuple: for data that should never change, such as coordinates, dates, or fixed settings.

Task 2.1

Create a tuple with your name, roll number, and age. Unpack it into three variables and print them in a sentence.

Part 3: Dictionaries

A dictionary stores data as **key : value** pairs. You look up a value using its key, not a position.

```
student = {
    "name": "Ali",
    "age": 20,
    "cgpa": 3.4
}

print(student["name"])          # Ali
print(student.get("city"))      # None (no error if the key is missing)
```

Adding, updating, and deleting

```
student["city"] = "Karachi"     # add a new key
student["age"] = 21             # update an existing key
del student["cgpa"]             # delete a key

print(student)
```

Looping through a dictionary

```
for key in student:
    print(key, "->", student[key])

for key, value in student.items():
    print(key, ":", value)

print(student.keys())           # all keys
print(student.values())         # all values
```

Checking for a key

```
if "name" in student:
    print("Name found")
```

Task 3.1

1. Create a dictionary for a book with the keys `title`, `author`, and `year`
2. Add a key `price`, change the `year`, and print every key and value using a loop
3. Ask the user for a key and print its value. If the key does not exist, print "Key not found"

Part 4: Sets

A set stores **unique** values only. It has no order and does not allow duplicates. Sets use curly braces.

```
numbers = {1, 2, 3, 3, 2, 1}
print(numbers)                  # {1, 2, 3}

numbers.add(4)
numbers.remove(1)
print(numbers)                  # {2, 3, 4}
```

Removing duplicates from a list

```
roll_numbers = [101, 102, 101, 103, 102]
unique = list(set(roll_numbers))
print(unique)                    # [101, 102, 103]
```

Set operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)        # union: {1, 2, 3, 4, 5, 6}
print(a & b)        # intersection: {3, 4}
print(a - b)        # difference: {1, 2}
```

Task 4.1

Two students list their favourite subjects. Using sets, print the subjects they **both** like and the subjects that **only the first student** likes.

Part 5: List Comprehension

A list comprehension builds a new list in one line.

```
# Normal way
squares = []
for i in range(1, 6):
    squares.append(i * i)

# List comprehension
squares = [i * i for i in range(1, 6)]
print(squares)                   # [1, 4, 9, 16, 25]
```

With a condition

```
evens = [i for i in range(1, 11) if i % 2 == 0]
print(evens)                     # [2, 4, 6, 8, 10]

names = ["ali", "sara", "omar"]
upper_names = [n.upper() for n in names]
print(upper_names)               # ['ALI', 'SARA', 'OMAR']
```

Task 5.1

1. Create a list of the cubes of numbers from 1 to 10
2. From the list `[12, 7, 25, 30, 3, 18]`, create a new list containing only the numbers greater than 10

Part 6: Functions

A function is a reusable block of code that performs one task. You write it once and call it as many times as you need.

Defining and calling a function

```
def greet():
    print("Hello, welcome to the AI lab!")

greet()
greet()
```

Parameters and return values

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)                    # 8
```

`print()` only shows a value on the screen. `return` sends the value back so you can store or reuse it.

Default parameters

```
def greet_user(name, greeting="Hello"):
    print(greeting, name)

greet_user("Ali")                # Hello Ali
greet_user("Sara", "Welcome")    # Welcome Sara
```

Returning more than one value

```
def min_max(numbers):
    return min(numbers), max(numbers)

smallest, largest = min_max([4, 9, 1, 7])
print(smallest, largest)         # 1 9
```

Variable scope

A variable created inside a function exists **only inside** that function.

```
def show():
    message = "inside"
    print(message)

show()
# print(message)                 # ERROR: message is not defined outside the function
```

Lambda functions

A lambda is a small one-line function without a name.

```
square = lambda x: x * x
print(square(5))                 # 25
```

```
add = lambda a, b: a + b
print(add(2, 3))                # 5
```

Task 6.1

1. Write a function `is_even(n)` that returns `True` if `n` is even, otherwise `False`
2. Write a function `average(numbers)` that takes a list and returns its average
3. Write a lambda that returns the cube of a number

Part 7: Lab Project – Student Record Manager

Goal: Build a menu-driven program that stores student records in a dictionary and uses a separate function for each action.

Data design: The key is the roll number. The value is another dictionary with the student's name and marks.

```
# students = {101: {"name": "Ali", "marks": 85}, 102: {...}}
students = {}

def add_student(roll, name, marks):
    students[roll] = {"name": name, "marks": marks}
    print("Student added.")

def show_all():
    if len(students) == 0:
        print("No records found.")
        return
    for roll, data in students.items():
        print(roll, "|", data["name"], "|", data["marks"])

def search_student(roll):
    if roll in students:
        data = students[roll]
        print("Name:", data["name"], "| Marks:", data["marks"])
    else:
        print("Student not found.")

def delete_student(roll):
    if roll in students:
        del students[roll]
        print("Student deleted.")
    else:
        print("Student not found.")

def average_marks():
    if len(students) == 0:
        return 0
    total = 0
    for data in students.values():
        total = total + data["marks"]
    return total / len(students)
```

```
# Main menu
while True:
    print("\n=== Student Record Manager ===")
    print("1. Add student")
    print("2. Show all students")
    print("3. Search student")
    print("4. Delete student")
    print("5. Show average marks")
    print("6. Exit")
    choice = input("Choose an option (1-6): ")

    if choice == "1":
        roll = int(input("Roll number: "))
        name = input("Name: ")
        marks = float(input("Marks: "))
        add_student(roll, name, marks)
    elif choice == "2":
        show_all()
    elif choice == "3":
        roll = int(input("Roll number to search: "))
        search_student(roll)
    elif choice == "4":
        roll = int(input("Roll number to delete: "))
        delete_student(roll)
    elif choice == "5":
        print("Average marks =", average_marks())
    elif choice == "6":
        print("Goodbye!")
        break
    else:
        print("Invalid option, try again.")
```

Task 7.1 (extend the project)

- Add an option to **update** a student's marks
- Add an option that prints the **name of the student with the highest marks**

Common Errors (Cheat Sheet)

Error	Cause	Fix
`IndexError`	Index is outside the list (for example `numbers[10]` in a 5-item list)	Check `len()` first
`KeyError`	Dictionary key does not exist	Use `.get(key)` or check `key in dict`
`TypeError` on a tuple	Trying to change a tuple item	Use a list if you need to change it

Error	Cause	Fix
`NameError` inside a function	Using a variable that exists only elsewhere	Pass it as a parameter
Function prints `None`	You used `print` inside the function instead of `return`	Use `return` to send the value back

---

Lab Tasks for Student Practice

Solve the following 5 tasks on your own. Paste your code and a screenshot of the output for each task in your lab file.

Task 1: Shopping List (Easy)

Create an empty list. Use a loop to take 5 items from the user and add them to the list. Then print the list sorted alphabetically and the total number of items.

Task 2: Word Counter (Easy)

Ask the user to type a sentence. Using a dictionary, count how many times each word appears and print the result.

Sample: Input `the cat and the dog` → Output `{'the': 2, 'cat': 1, 'and': 1, 'dog': 1}`

Hint: Use `sentence.split()` to break the sentence into words.

Task 3: Unique Marks (Medium)

The list `[70, 85, 70, 90, 85, 60, 90]` contains repeated marks. Using a set, print the unique marks in **descending** order, and print how many duplicates were removed.

Task 4: Functions Toolkit (Medium)

Write these functions and test each one at least twice:

1. `is_prime(n)` returns `True` if `n` is a prime number
2. `factorial(n)` returns the factorial of `n`
3. `count_vowels(text)` returns the number of vowels in a string

Task 5: Contact Book (Challenging)

Build a menu-driven contact book using a dictionary (name → phone number) and functions. It must allow the user to:

1. Add a contact
2. Search a contact by name
3. Delete a contact
4. Show all contacts in alphabetical order of name
5. Exit

Submission Instructions

- Paste the **code and output** of every task in your lab file
- Write **comments** in your code wherever needed
- Name your notebook: `Lab2_RollNumber.ipynb`
- Complete **Tasks 1 to 5** before the next lab

End of Lab 02. Next lab: OOP, Files and Error Handling